



---

# LifeData Analytics Hub

Admin Manual

Prepared by: Lifewood PH Team

Date: July 25, 2026

# Contents

---

- 1.0 System Overview ..... 3
- 2.0 Supabase Configuration ..... 5
- 3.0 Adding New Dashboards ..... 7
- 4.0 Environment Variables ..... 10
- 5.0 Deployment ..... 12
- 6.0 Security ..... 13

---

## How to use this document

Audience: Administrators managing accounts, configuration, and environment settings for this platform.

Conventions: monospace text = literal commands or code you type; < angle brackets > = replace with your value; callout boxes highlight warnings or prerequisites.

# System Overview

LifeData is a React 19 SPA with Vite 7 build tooling, deployed to Vercel. Backend: Supabase (PostgreSQL) with Row-Level Security for data access control. Authentication uses Supabase Auth with email/password login.

The application uses a config-driven dashboard system – new dashboards can be added without writing component code. This manual covers configuration, deployment, and security for administrators maintaining the platform.

| Component | Technology            | Purpose                                       |
|-----------|-----------------------|-----------------------------------------------|
| Frontend  | React 19 + TypeScript | SPA with config-driven dashboards             |
| Build     | Vite 7                | Fast dev server + optimized production builds |
| Database  | Supabase / PostgreSQL | Data storage with RLS                         |
| Auth      | Supabase Auth         | Email/password login                          |
| Hosting   | Vercel                | Continuous deployment from GitHub             |
| Config    | tableDashboards.ts    | Declarative dashboard definitions             |

## Supabase Configuration

---

Create a Supabase project at [supabase.com](https://supabase.com) and note your project URL and anon key. These go into your .env file (see Section 4.0).

Tables must match the expected column schema. The standard schema includes: firstName, lastName, gender, affiliation, email, phone, joinedDate, age.

### Prerequisite

Enable Row-Level Security on EVERY table. Without RLS, all data is publicly readable via the anon key.

SQL – ANON READ POLICY

```
CREATE POLICY "Allow anon read" ON "TableName" FOR SELECT TO anon USING (true);
```

SQL – AUTH USER FULL ACCESS POLICY

```
CREATE POLICY "Allow all for authenticated" ON "TableName" FOR ALL TO authenticated USING (true);
```

Apply these two policies per table. The first allows anonymous read access (needed for public dashboards). The second lets authenticated users insert/update/delete.

## Adding New Dashboards

Adding a new dashboard requires zero component code. The GenericTableDashboard auto-renders from configuration.

- 1. Create the Supabase table and upload your CSV/XLSX data.
- 2. Run the two RLS policies from Section 2.0 on the new table.
- 3. Add a TableDashboardConfig entry in src/config/tableDashboards.ts.
- 4. No component code changes needed — GenericTableDashboard auto-renders the new dashboard.
- 5. Optional: add preFilter for row-level filtering by a specific column/value.

Each config entry requires these fields:

| Field         | Type           | Required | Description                       |
|---------------|----------------|----------|-----------------------------------|
| tabId         | string         | Yes      | Unique tab identifier             |
| tableId       | string         | Yes      | Supabase table name               |
| label         | string         | Yes      | Display name in sidebar           |
| sidebarFolder | string         | No       | Folder grouping in nav            |
| flagCode      | string         | No       | Country/region filter code        |
| title         | string         | Yes      | Dashboard heading                 |
| subtitle      | string         | No       | Subtitle below heading            |
| columns       | ColumnConfig[] | Yes      | Column definitions                |
| preFilter     | object         | No       | Row-level filter: {column, value} |

# Environment Variables

| Variable               | Description          | Source                              |
|------------------------|----------------------|-------------------------------------|
| VITE_SUPABASE_URL      | Supabase project URL | Supabase dashboard > Settings > API |
| VITE_SUPABASE_ANON_KEY | Public anon key      | Supabase dashboard > Settings > API |

**Warning**

Never commit .env files to version control. Never expose the service\_role key client-side. The anon key is safe for browser use — it is gated by RLS.

For local development, copy .env.example to .env and fill in the values. For Vercel deployment, set these in the Vercel dashboard under Project Settings > Environment Variables.

.ENV TEMPLATE

```
VITE_SUPABASE_URL="https://your-project.supabase.co"
VITE_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ..."
```

## Deployment

---

The application is continuously deployed to Vercel from the GitHub repository. Every push to the main branch triggers an automatic deployment.

1. Run `npm run lint` to verify code quality (fix any errors).
2. Run `npm run build` to produce a production build. Output goes to `dist/`.
3. Commit and push to main. Vercel auto-deploys.
4. Verify the deployment in Vercel dashboard.

### COMMANDS

```
npm run lint
npm run build
npm run preview # serve production build locally
```

The build pipeline runs TypeScript type checking as part of the build step. There is no Dockerfile or separate CI/CD pipeline – Vercel handles all build and deployment.

6.0

## Security

| Measure            | Implementation                           | Criticality |
|--------------------|------------------------------------------|-------------|
| Row-Level Security | Per-table SELECT/ALL policies            | Critical    |
| Auth               | Supabase Auth, email/password            | Required    |
| Secrets            | .env never committed, Vercel env vars    | Required    |
| Key separation     | anon key = public, service_role = server | Critical    |
| HTTPS              | Vercel edge network (default)            | Automatic   |
| Etag caching       | Vercel edge (default)                    | Automatic   |

**Warning**

The service\_role key bypasses RLS entirely. It must NEVER be exposed in client-side code or committed to version control.

Before going live with a new table, verify that appropriate SELECT policies exist. Test by loading the dashboard page while logged out – if data appears without auth, the anon SELECT policy is working correctly. If sensitive data should NOT be public, omit the anon policy.

Etag-based caching and HTTPS are handled automatically by Vercel's edge network. No additional configuration is needed.